RÉPUBLIQUE DU SÉNÉGAL

Un Peuple — Un But — Une Foi



UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR

Faculté des Sciences et Techniques

Département de Mathématiques et Informatique

Master Mathématiques, Cryptographie et Sécurité — TDSI

MémoTDSI

La mémothèque de la filière TDSI

Texte de l'oral de soutenance — 37 diapositives, 6 orateurs

Équipe : M1 M2 M3 M4 M5 M6

Année universitaire : 2026–2027

Durée : environ 25 minutes de présentation et 8 minutes de démonstration

Organisation

Les six orateurs se partagent les 37 diapos. Chacun termine par une phrase de passage qui annonce le suivant. Le texte est fait pour être dit à voix haute : on peut le reformuler avec ses mots, mais on garde les chiffres exacts (225 tests, 113 classes, 600 000 itérations, 80 bits).

Orateur	Diapos	Thème	Durée
M1	1 à 5	Présentation, circuit, espace étudiant	4 min
M2	6 à 10	Enseignant, planification, PV, convocations	4 min
M3	11 à 17	Nouveautés, pile technique, architecture	4 min
M4	18 à 24	Le code Java : service, objets, JDBC, JavaFX	5 min
M5	25 à 29	Cryptographie et protection des données	4 min
M6	30 à 37	QR codes, TOTP, PDF, tests, démonstration	4 min + démo

M1 — L'application

Diapos 1 à 5

Présentation, acteurs, circuit, portail, espace étudiant

Diapo 1 — MémoTDSI

« Bonjour. Nous sommes l'équipe M1 à M6 du Master Mathématiques, Cryptographie et Sécurité. Nous présentons MémoTDSI, la mémothèque de la filière TDSI. C'est une application de bureau qui suit un mémoire de bout en bout, du dépôt par l'étudiant jusqu'à sa publication avec un procès-verbal signé électroniquement. Comme nous sommes en master sécurité, la sécurité a été pensée dès le départ, pas ajoutée à la fin. »

Diapo 2 — Trois acteurs, un circuit

« Trois acteurs : l'étudiant, l'enseignant et la scolarité. Comme dans le vrai système d'information d'une université, chacun ne fait que ce qui relève de son rôle. L'étudiant dépose, l'encadreur donne son avis, et la scolarité organise le reste : la campagne, les soutenances et les procès-verbaux. »

Diapo 3 — Le circuit d'un mémoire

« Quatre étapes. Un : l'étudiant dépose son PDF, qui est contrôlé, chiffré, et dont on calcule l'empreinte SHA-256. Deux : l'encadreur donne un avis favorable ou demande des corrections. Trois : la scolarité planifie la soutenance. Quatre : après la soutenance, la note donne la mention et le PV est signé. En cas de corrections, ou si la note est inférieure à 10, on revient à l'étudiant. L'étudiant ne fixe ni la date ni la mention. »

Diapo 4 — Un portail universitaire

« L'interface ressemble à un portail d'université : photo du campus à la connexion, menu avec des icônes, cloche de notifications et textes en grand. Le tableau de bord dit à chacun ce qu'il doit faire maintenant, par exemple « 2 mémoires à évaluer ». Pour la démonstration, les comptes de test se remplissent en un clic. »

Diapo 5 — Espace étudiant

« L'étudiant suit son mémoire étape par étape : une pastille de couleur montre l'état, avec la date de soutenance et le résultat. Il imprime son attestation de dépôt, qui porte l'empreinte SHA-256 de son PDF, et sa convocation officielle. »

Passage : « Je laisse la parole à M2, qui va vous montrer le côté enseignant et scolarité. »

M2 — Enseignants et scolarité

Diapos 6 à 10

Enseignant, planification, procès-verbal, convocations, pilotage

Diapo 6 — Espace enseignant

« L'encadreur ne note pas. Il autorise la soutenance, ou demande des corrections avec un motif obligatoire. Avant toute décision, l'application vérifie que le PDF n'a pas été modifié. Dans « Mes jurys », l'enseignant voit son rôle dans chaque soutenance (président, examinateur ou encadreur) et imprime sa convocation. »

Diapo 7 — La scolarité planifie

« La scolarité choisit la date, la salle et le jury. Avant de valider, le service vérifie les règles : un horaire entre 7 h et 20 h, au moins un examinateur, et l'encadreur ne peut pas présider. Il vérifie aussi qu'aucune salle et qu'aucun enseignant ne sont pris deux fois au même moment. L'étudiant et chaque membre du jury sont ensuite notifiés. »

Diapo 8 — Le procès-verbal

« Après la soutenance, la scolarité saisit la note et la mention s'affiche en direct, selon le barème LMD. Pour signer le PV, il faut retaper son mot de passe. Avec 10 ou plus, le mémoire est publié ; en dessous, il est ajourné et revient à l'étudiant. La diffusion peut être libre, restreinte, ou confidentielle avec un embargo. »

Diapo 9 — Les convocations

« Pour chaque soutenance, l'application rédige une lettre officielle par destinataire : l'étudiant, le président, l'examinateur et l'encadreur. Elle porte l'en-tête de l'UCAD, le tableau de la soutenance et la place pour le cachet. La scolarité imprime toutes les lettres ; chaque autre personne ne peut obtenir que la sienne. »

Diapo 10 — Pilotage

« La scolarité voit tous les mémoires d'un coup d'œil : des compteurs par étape, des filtres et des pastilles de couleur. »

Passage : « M3 va maintenant vous présenter deux nouveautés et la façon dont le code est organisé. »

M3 — Nouveautés et architecture

Diapos 11 à 17

Campagne par niveau, import Excel, pile technique, organisation, architecture

Diapo 11 — Partie 2 (intercalaire)

« Voici deux nouveautés demandées par la scolarité. »

Diapo 12 — Campagne de dépôt par niveau

« Chaque niveau, de L1 à M2, a maintenant sa propre campagne. La scolarité l'ouvre ou la ferme, fixe une date limite, et la prolonge ou l'écourte avec les boutons +7 et −7 jours. Les étudiants du niveau sont prévenus à chaque changement, par exemple : « date limite du niveau M2 prolongée du 30/11 au 07/12 ». Dans le code, une `EnumMap<Niveau, Campagne>` contient la campagne de chaque niveau. Au passage, nous avons corrigé un bug : le sélecteur de date de JavaFX n'enregistrait une date tapée au clavier qu'après un appui sur Entrée. »

Diapo 13 — Comptes et import Excel

« Il y a trois types de comptes : étudiant, enseignant et personnel administratif, chacun avec son téléphone. La scolarité peut importer un fichier Excel entier ; chaque ligne donne un résultat : créé, réinscrit, mis à jour ou erreur. Le point intéressant : un fichier `.xlsx` n'est qu'une archive ZIP de fichiers XML. Nous le lisons avec le JDK, sans Apache POI, et nous nous protégeons contre la bombe ZIP et contre l'attaque XXE. »

Diapo 14 — Partie 3 (intercalaire)

« Passons sous le capot : comment l'application est construite en Java. »

Diapo 15 — La pile technique

« Java 21, JavaFX 21 pour l'interface, SQLite avec JDBC pour les données, Maven pour construire et JUnit 5 pour tester. Seulement 4 dépendances : tout le reste est écrit par nous avec le JDK, y compris la cryptographie, la lecture Excel, les QR codes et les PDF. Le projet compte 113 classes, environ 15 500 lignes et 225 tests. »

Diapo 16 — Organisation du code

« Un paquet par couche : `controller` pour un contrôleur par écran, `service` pour les règles métier, `dao` pour l'accès à la base, `model` pour les objets et `util` pour les outils communs. Une couche n'appelle que celle du dessous ; un écran ne touche donc jamais la base. »

Diapo 17 — Architecture

« C'est le schéma MVC, complété par une couche Service et une couche DAO. Le point le plus important est en rouge : les droits sont vérifiés dans les services. Cacher un bouton ne suffit pas ; si quelqu'un appelait la méthode autrement, le service refuserait quand même. »

Passage : « M4 va vous montrer le code Java lui-même, à partir d'un exemple réel. »

M4 — Le code Java

Diapos 18 à 24

Parcours d'un clic, service, modèle objet, DAO, JavaFX, threads, exceptions

Diapo 18 — Le parcours d'un clic

« Prenons un exemple concret : la scolarité planifie une soutenance. Le clic sur OK arrive au contrôleur, qui appelle le service. Le service vérifie le droit, puis les règles. Le DAO cherche les conflits en SQL, puis enregistre tout dans une seule transaction, et les notifications partent. Si une règle est violée, une exception remonte : le message s'affiche, la fenêtre reste ouverte et rien n'est enregistré. »

Diapo 19 — Le code du service

« Voici le vrai code de `planifier`. La première ligne est `exigerAdmin()`, le contrôle du rôle. Chaque règle lève ensuite une exception avec un message clair. Le statut du mémoire est un enum : seul un mémoire `AUTORISE` peut être programmé. Comme le service ne dépend pas de l'écran, nos tests l'appellent directement, avec chaque rôle. »

Diapo 20 — Modèle objet et Java moderne

« Les états sont des enums, comme `StatutMemoire` : le compilateur vérifie les valeurs, ce qui évite les fautes de frappe dans des chaînes. Les valeurs immuables sont des records, comme `Campagne`. Nous utilisons aussi les nouveautés de Java : switch expressions, pattern matching avec `instanceof`, et text blocks pour écrire le SQL. »

Diapo 21 — Accès aux données : DAO + JDBC

« `GenericDAO<T>` est une interface générique qui regroupe les opérations communes. Le point à retenir, c'est la transaction : `setAutoCommit(false)`, puis `commit`, ou `rollback` en cas d'erreur. C'est tout ou rien. Toutes les requêtes passent par `PreparedStatement`, donc aucune injection SQL n'est possible. Le schéma de la base évolue par migrations numérotées, de V2 à V10. »

Diapo 22 — L'interface JavaFX

« Le fichier FXML décrit l'écran. Le contrôleur reçoit automatiquement les composants grâce à `@FXML` : un `fx:id` dans le FXML correspond à un champ dans le contrôleur. Le style tient dans une feuille CSS à jetons de couleur, et le thème sombre ne redéfinit que ces jetons. »

Diapo 23 — Ne jamais figer l'interface

« JavaFX n'a qu'un seul fil pour l'affichage. Un calcul long, comme un PBKDF2 à 600 000 itérations, figerait la fenêtre. Notre classe `TacheFond` fait donc ce calcul sur un autre fil, puis revient sur le fil JavaFX pour afficher le résultat. Pendant le calcul, les boutons sont grisés, ce qui évite un double clic. »

Diapo 24 — Les erreurs, côté Java

« Chaque exception a un sens. `IllegalArgumentException` : une saisie invalide. `IllegalStateException` : une étape du circuit non respectée. `SecurityException` : un droit refusé. `SQLException` : une erreur technique. Dans les fenêtres, un multi-catch affiche le message, et `ev.consume()` garde la fenêtre ouverte. »

Passage : « M5 va maintenant vous expliquer comment la cryptographie protège les données. »

M5 — La cryptographie

Diapos 25 à 29

Cryptographie du JDK, protection des données, signature, falsification, journal

Diapo 25 — La cryptographie avec le JDK

« Aucune bibliothèque externe : l'API standard du JDK suffit. PBKDF2 protège les mots de passe, AES-GCM chiffre les fichiers, HMAC chaîne le journal et Ed25519 signe les documents. Les clés sont rangées dans un coffre PKCS12, séparé de la base. »

Diapo 26 — Protection des données

« Le PDF déposé est chiffré en AES-256-GCM. La clé est dans un coffre séparé, et l'empreinte SHA-256 est vérifiée à chaque ouverture. Le mot de passe est haché avec PBKDF2, 600 000 itérations et un sel de 16 octets. »

Diapo 27 — Signature du procès-verbal

« Le PV est signé avec Ed25519. La signature couvre l'empreinte du PDF, la note, la mention, la date et le signataire. La clé privée de la scolarité est chiffrée par son mot de passe : pour signer, il faut le retaper. »

Diapo 28 — Falsifier un résultat ? Détecté

« C'est notre démonstration préférée. Si l'on change la mention directement dans la base, par exemple « Bien » en « Très bien », la fiche affiche aussitôt « signature invalide ». »

Diapo 29 — Un journal infalsifiable

« Chaque action sensible est journalisée. Chaque ligne est liée à la précédente par un HMAC. Si l'on modifie, insère ou supprime une ligne, la vérification de la chaîne le détecte. »

Passage : « M6 va présenter nos dernières fonctions de confiance, puis lancer la démonstration. »

M6 — Confiance et démonstration

Diapos 30 à 37

Documents vérifiables, encodeur QR, TOTP, PDF, tests, qualité, démonstration

Diapo 30 — Partie 4 (intercalaire)

« Dernière partie : des documents qu'on peut vérifier, et une connexion renforcée. »

Diapo 31 — Documents vérifiables

« Une convocation imprimée se falsifie facilement. Chaque convocation, PV ou attestation porte donc un QR code qui contient ses informations et leur signature Ed25519, ainsi qu'un code de 16 caractères imprimé à côté. L'écran « Vérifier un document » est accessible même sans compte. Il répond : authentique, plus valable (par exemple si la soutenance a été déplacée), falsifié ou inconnu. »

Diapo 32 — Un encodeur QR écrit en Java

« Nous avons écrit l'encodeur QR nous-mêmes, d'après la norme ISO 18004. Il protège les données par un code correcteur de Reed-Solomon dans le corps fini $\mathrm{GF}(2^8)$, ce qui est directement notre cours. Il est validé sur 67 QR codes, identiques module par module à une implémentation de référence. »

Diapo 33 — Double authentification TOTP

« Après le mot de passe, l'application demande un code à 6 chiffres, comme Google Authenticator, qui change toutes les 30 secondes. Ce code est un HMAC-SHA1 du secret et de l'heure. Un code ne sert qu'une fois, et cinq codes faux verrouillent le compte. Nous l'avons testé avec les vecteurs officiels de la RFC 6238. »

Diapo 34 — Export PDF sans bibliothèque

« Les aperçus ont un bouton « Enregistrer en PDF ». Nous écrivons le fichier PDF nous-mêmes, objet par objet, sans iText ni PDFBox, et un test vérifie que sa structure est correcte. »

Diapo 35 — Stratégie de tests

« 225 tests JUnit 5, chacun sur une base SQLite jetable. Ils couvrent le circuit complet, les droits de chaque rôle, la falsification, les vecteurs officiels des normes et l'import Excel. GitHub Actions les relance à chaque push. »

Diapo 36 — Qualité

« En résumé : 225 tests, aucune bibliothèque de cryptographie externe, trois rôles strictement séparés, et un thème clair ou sombre. »

Diapo 37 — Merci

« Merci de votre attention. Place à la démonstration. »

Démonstration (environ 8 minutes)

1. Babacar NDIAYE (`babs@mcs` / `babs123`) : sa convocation avec QR code, puis « Enregistrer en PDF ».
2. M. SAMB (`samb@mcs` / `samb123`) : donne son avis sur un mémoire.
3. La scolarité (`admin@mcs` / `admin123`) : +7 jours sur la campagne M2, planification d'une soutenance, saisie du PV signé.
4. « Vérifier un document » : coller le texte du QR, résultat authentique ; changer une lettre du nom, résultat falsifié.
5. « Mon profil » : activer la double authentification avec un téléphone, se déconnecter, puis se reconnecter avec le code.
6. Modifier la mention dans la base SQLite : la fiche affiche « signature invalide ».

Questions probables du professeur de Java

Chaque question a un répondant désigné : celui qui a présenté la diapo concernée.

Question	Réponse courte	Répond
Pourquoi pas Spring ou Hibernate ?	Pour une application de bureau sur un seul poste, JDBC et des couches écrites à la main suffisent, et montrent qu'on comprend ce que ferait un framework.	M3
À quoi sert le rollback ?	Sans lui, on pourrait enregistrer une soutenance sans son jury. Avec, c'est tout ou rien.	M4
Pourquoi un enum plutôt qu'une chaîne ?	Le compilateur vérifie les valeurs ; une faute de frappe devient une erreur de compilation, pas un bug.	M4
Que se passe-t-il si on calcule sur le fil JavaFX ?	La fenêtre se fige pendant le calcul. C'est pour ça que `TacheFond` utilise un autre fil.	M4
Comment empêchez-vous l'injection SQL ?	`PreparedStatement` partout, et un tri choisi dans une liste fermée (un enum).	M4
Où sont vérifiés les droits ?	Dans les services, pas dans les écrans. Les tests appellent les services avec un rôle non autorisé et attendent une `SecurityException`.	M3
Pourquoi AES-GCM et pas AES-CBC ?	GCM chiffre et authentifie : toute modification du fichier fait échouer le déchiffrement.	M5
Pourquoi 600 000 itérations de PBKDF2 ?	C'est la recommandation actuelle de l'OWASP pour PBKDF2-HMAC-SHA256 : chaque essai coûte cher à un attaquant.	M5
Comment savez-vous que votre QR code est juste ?	67 codes comparés module par module à une implémentation de référence, plus trois matrices de référence dans les tests.	M6
Comment empêcher de réutiliser un code TOTP ?	On mémorise le dernier pas de 30 secondes accepté ; un code d'un pas égal ou plus ancien est refusé.	M6